

Auto Scaling

1. Create one VPC in N. Virginia region.
2. Create two subnets: one public subnet and one private subnet.
3. Attach an IGW to the VPC.

Step 1: Login to AWS Console

1. Open the AWS Management Console.
2. Sign in with your AWS account.
3. From the Region selector (top-right corner), select **US East (N. Virginia) (us-east-1)**.

Expected Output:

- You are working in the **N. Virginia** region.
-

Step 2: Open the VPC Dashboard

1. In the search bar, type **VPC**.
2. Open the **VPC** service.

Expected Output:

- The VPC Dashboard opens.
-

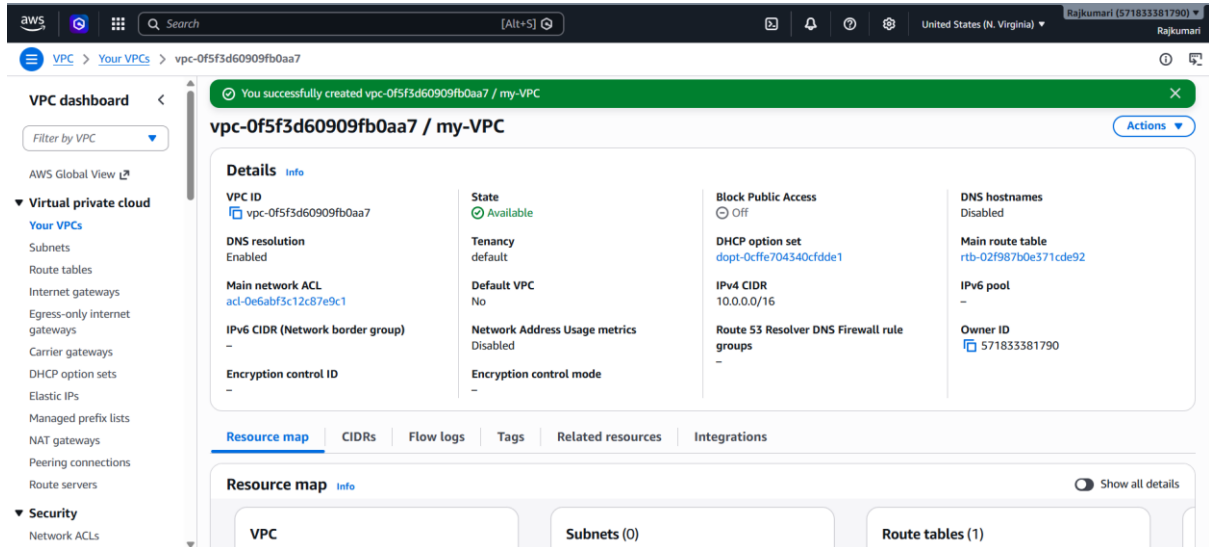
Step 3: Create a Custom VPC

1. In the left navigation pane, click **Your VPCs**.
2. Click **Create VPC**. Choose:

Field	Value
Resources to create VPC only	
Name	My-VPC
IPv4 CIDR	10.0.0.0/16

Field	Value
IPv6 CIDR	No IPv6 CIDR Block
Tenancy	Default

3. Click **Create VPC**.



Step 4: Create the Public Subnet

1. Click **Subnets**.
2. Click **Create subnet**.

Enter:

Field	Value
VPC	My-VPC
Subnet Name	Public-Subnet
Availability Zone	us-east-1a
IPv4 CIDR	10.0.1.0/24

Click **Create subnet**.

Enable Auto-Assign Public IP

1. Select **Public-Subnet**.

2. Click **Actions** → **Edit subnet settings**.
3. Enable **Auto-assign public IPv4 address**.
4. Click **Save**.

Expected Output:

- Instances launched in this subnet automatically receive a public IPv4 address.
-

Step 5: Create the Private Subnet

1. Click **Create subnet** again.

Enter:

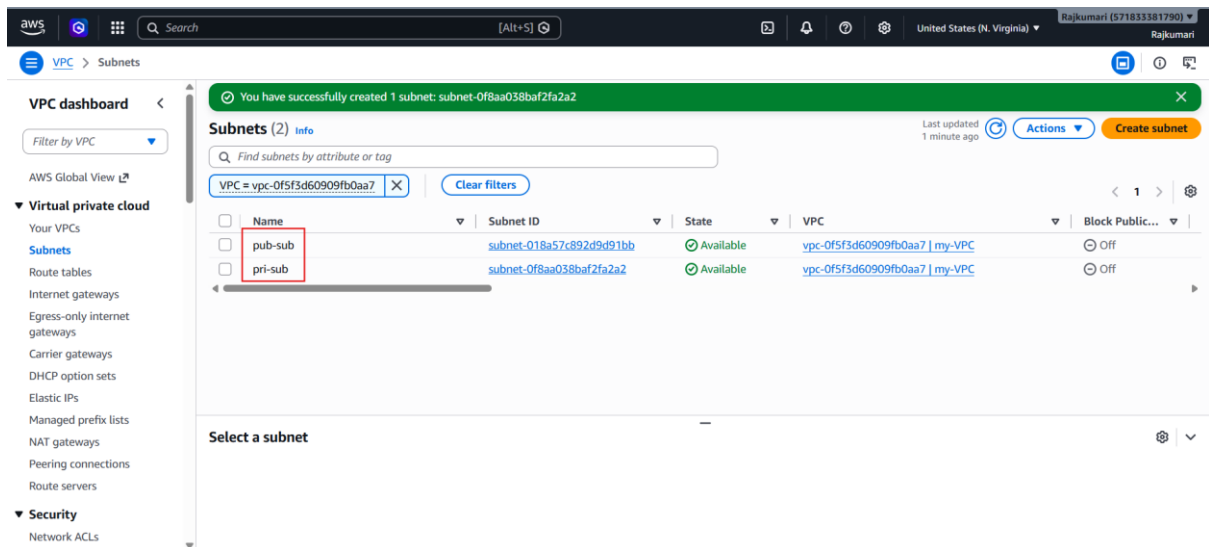
Field	Value
VPC	My-VPC
Subnet Name	Private-Subnet
Availability Zone	us-east-1b
IPv4 CIDR	10.0.2.0/24

Click **Create subnet**.

Do not enable Auto-assign Public IP for the private subnet.

Expected Output:

- A private subnet is created.



Step 6: Create an Internet Gateway (IGW)

1. In the left navigation pane, click **Internet Gateways**.
2. Click **Create Internet Gateway**.

Enter:

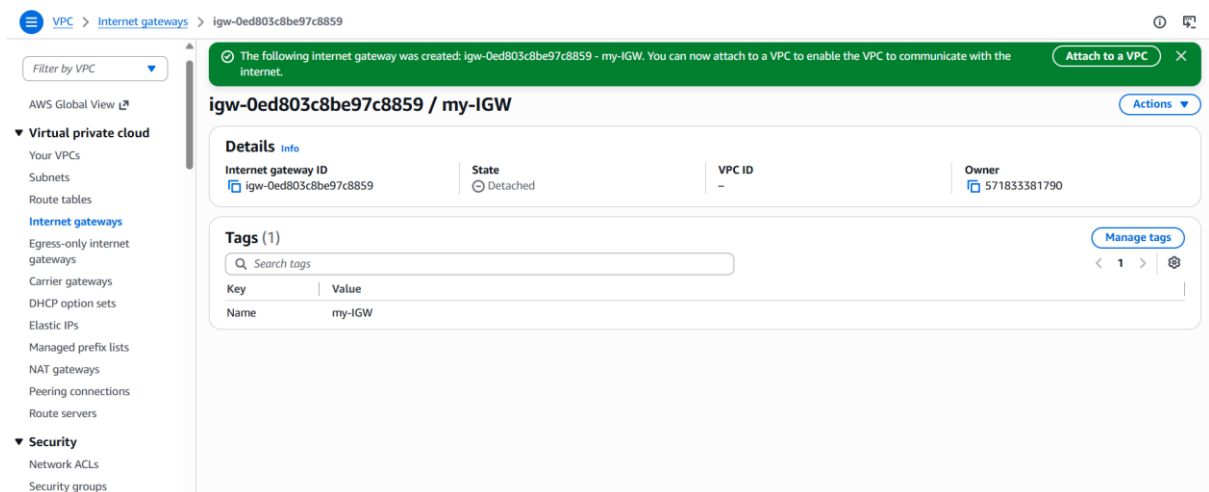
Field Value

Name My-IGW

Click **Create Internet Gateway**.

Expected Output:

- The Internet Gateway is created but is not yet attached to any VPC.

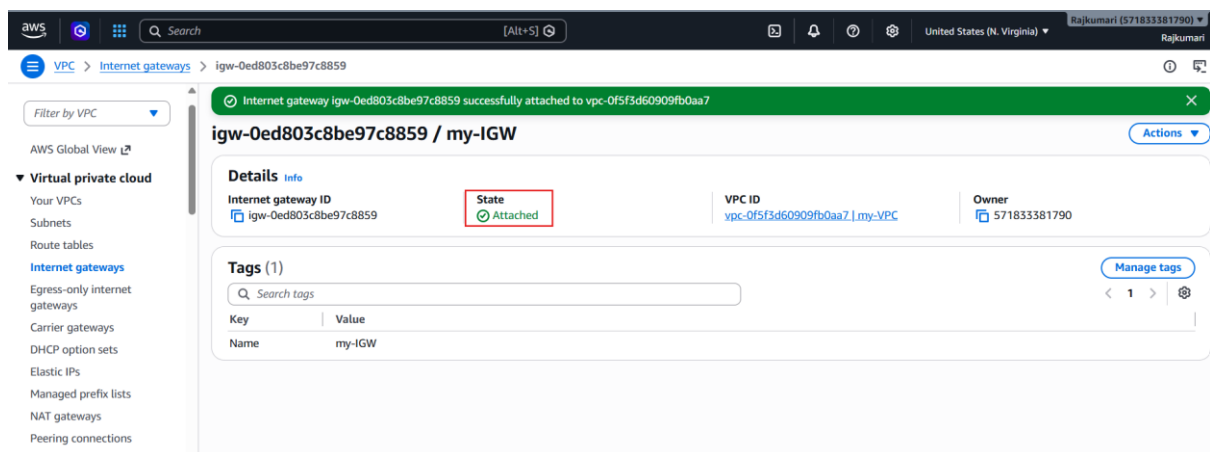


Step 7: Attach the Internet Gateway to the VPC

1. Select **My-IGW**.
2. Click **Actions** → **Attach to VPC**.
3. Choose **My-VPC**.
4. Click **Attach Internet Gateway**.

Expected Output:

- The Internet Gateway status changes to **Attached**.



4. Create one public route table (RT) and one private route table.

5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

Objective - Create:

- 1 Public Route Table
- 1 Private Route Table
- 1 NAT Gateway in the Public Subnet
- Associate the Public Route Table with the Public Subnet
- Associate the Private Route Table with the Private Subnet
- Configure the Private Route Table to use the NAT Gateway for outbound internet access

Step 1: Create the Public Route Table

1. Open the **VPC** service.
2. In the left menu, click **Route Tables**.

3. Click **Create route table**.

Enter the following:

Field	Value
-------	-------

Name	Public-RT
------	-----------

VPC	My-VPC
-----	--------

4. Click **Create route table**.

Expected Output:

- The Public Route Table is created.
-

Step 2: Add an Internet Route to the Public Route Table

1. Select **Public-RT**.
2. Go to the **Routes** tab.
3. Click **Edit routes**.
4. Click **Add route**.

Enter:

Destination	Target
-------------	--------

0.0.0.0/0	Internet Gateway (My-IGW)
-----------	---------------------------

5. Click **Save changes**.

Expected Output:

- The Public Route Table now sends internet traffic to the Internet Gateway.

Step 3: Associate the Public Route Table with the Public Subnet

1. Select **Public-RT**.
2. Go to the **Subnet Associations** tab.
3. Click **Edit subnet associations**.
4. Select **Public-Subnet**.

5. Click **Save associations**.

Expected Output:

- The Public Subnet is associated with the Public Route Table.

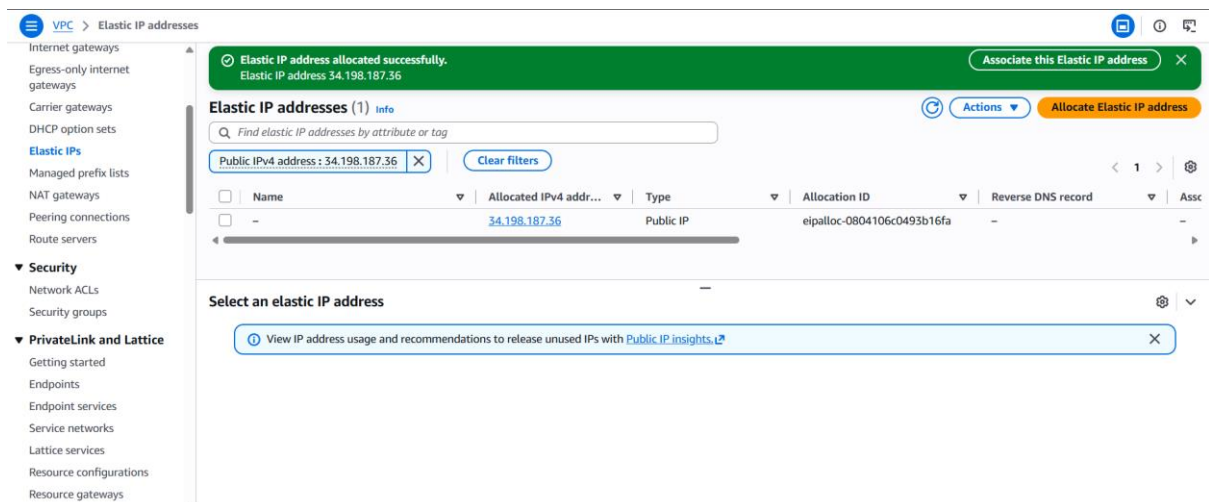
Step 4: Allocate an Elastic IP (EIP)

A NAT Gateway requires an Elastic IP.

1. In the VPC console, click **Elastic IPs**.
2. Click **Allocate Elastic IP address**.
3. Leave the default settings.
4. Click **Allocate**.

Expected Output:

- An Elastic IP is created.



Step 5: Create the NAT Gateway

1. In the VPC console, click **NAT Gateways**.
2. Click **Create NAT Gateway**.

Enter:

Field	Value
Name	My-NAT-Gateway

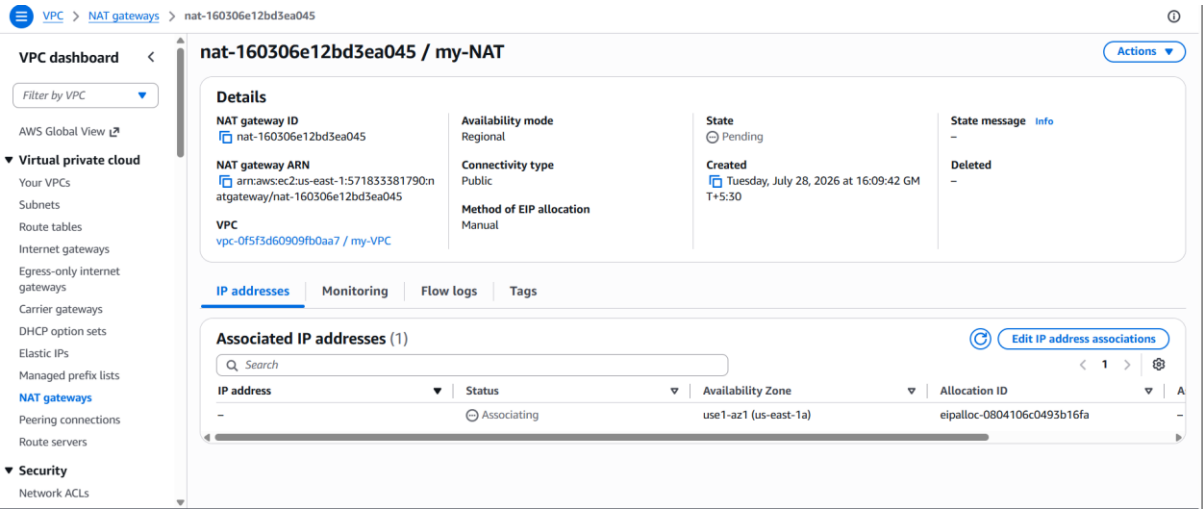
Field	Value
Subnet	Public-Subnet
Connectivity Type	Public
Elastic IP	Select the Elastic IP created above

3. Click **Create NAT Gateway**.

Wait until the **State** changes to **Available**.

Expected Output:

- The NAT Gateway is available in the Public Subnet.



Step 6: Create the Private Route Table

1. Click **Route Tables**.
2. Click **Create route table**.

Enter:

Field	Value
-------	-------

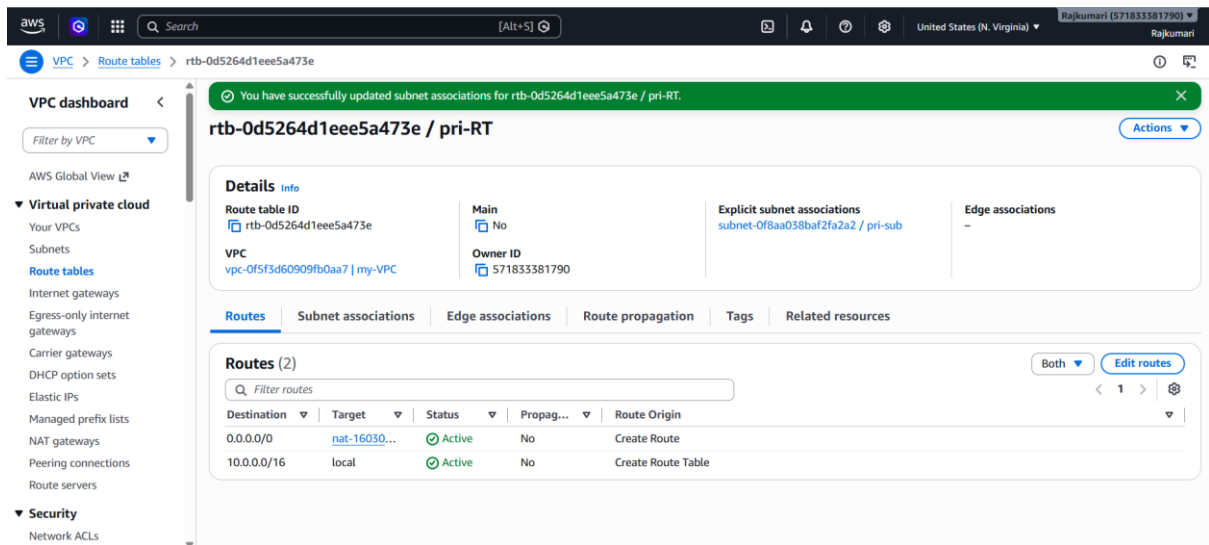
Name Private-RT

VPC My-VPC

3. Click **Create route table**.

Expected Output:

- The Private Route Table is created.



6.Create two instances, one in the public subnet and one in the private subnet.

7.Deploy Apache server on both EC2 instances with a sample index.html file.

Objective

- Create one EC2 instance in the **Public Subnet**.
- Create one EC2 instance in the **Private Subnet**.
- Install the Apache web server on both instances.
- Create a sample index.html page.
- Verify that Apache is running.

step 1: Launch the Public EC2 Instance

1. Open the EC2 Console.
2. Click **Launch Instance**.

Configure the instance

Field	Value
Name	Public-EC2

Field	Value
AMI	Amazon Linux 2023
Instance Type	t3.micro (Free Tier eligible)
Key Pair	Select your existing key pair
VPC	My-VPC
Subnet	Public-Subnet
Auto-assign Public IP	Enable
Security Group	Create new (Public-SG)

Security Group Rules (Public-SG)

Type Port Source

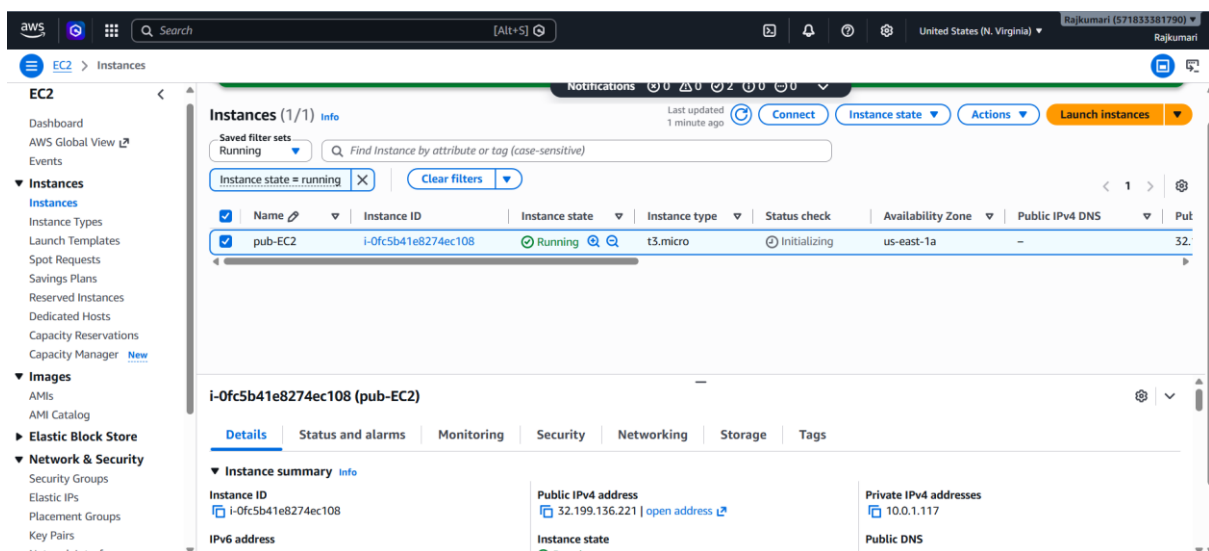
SSH 22 My IP (recommended)

HTTP 80 Anywhere (0.0.0.0/0)

3. Click **Launch Instance**.

Expected Output:

- Public EC2 is running with a **public IP address**.



Step 2: Launch the Private EC2 Instance

1. Click **Launch Instance** again.

Configure the instance

Field	Value
Name	Private-EC2
AMI	Amazon Linux 2023
Instance Type	t2.micro
Key Pair	Same key pair
VPC	My-VPC
Subnet	Private-Subnet
Auto-assign Public IP	Disable
Security Group	Create new (Private-SG)

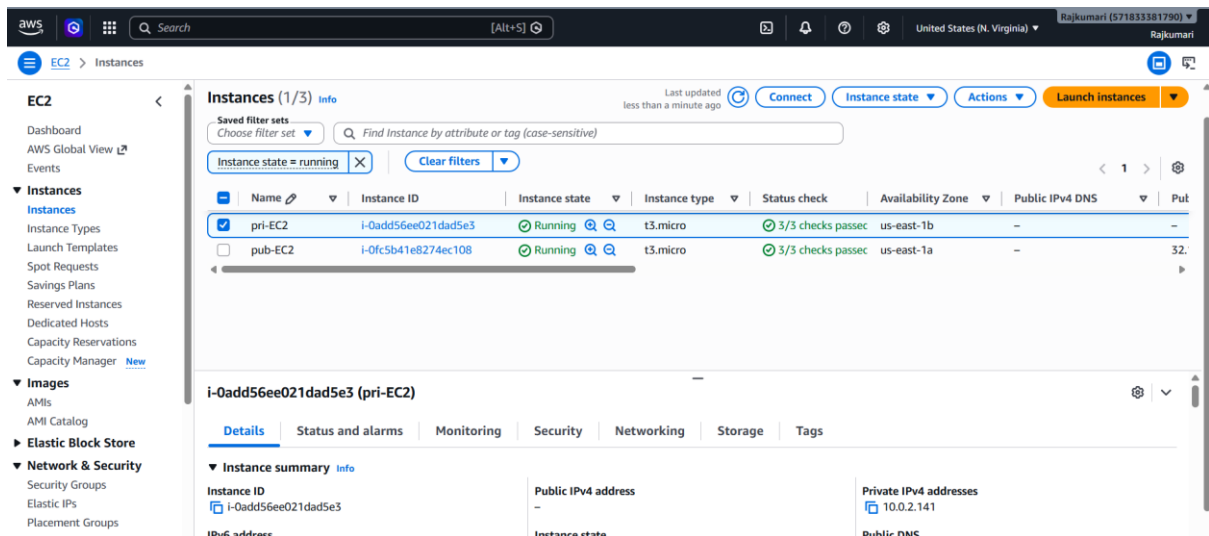
Security Group Rules (Private-SG)

Type	Port	Source
SSH	22	Public-SG (or the public instance's private IP/CIDR as per your lab design)
HTTP	80	Public-SG (or the ALB security group later in the project)

3. Click **Launch Instance**.

Expected Output:

- Private EC2 is running with only a **private IP address**.



Step 3: Connect to the Public EC2

From your local machine: `$ ssh -i mykey.pem ec2-user@<Public-IP>`

Step 4: Install Apache on the Public EC2

Run the following commands:

```
$ sudo dnf update -y
```

```
sudo dnf install httpd -y
```

```
$ sudo systemctl start httpd
```

```
$ sudo systemctl enable httpd
```

```
$ sudo systemctl status httpd
```

```
Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-ldap-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
httpd-2.4.68-1.amzn2023.0.1.x86_64
httpd-core-2.4.68-1.amzn2023.0.1.x86_64
httpd-filesystem-2.4.68-1.amzn2023.0.1.noarch
httpd-tools-2.4.68-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch
mod_http2-2.0.42-1.amzn2023.0.1.x86_64
mod_lua-2.4.68-1.amzn2023.0.1.x86_64

Complete!
[ec2-user@ip-10-0-1-117 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-1-117 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-10-0-1-117 ~]$
```

```

ec2-user@ip-10-0-1-117:~$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-07-28 11:02:48 UTC; 16min ago
     Docs: man:httpd.service(8)
   Main PID: 26345 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 177 (limit: 1059)
   Memory: 14.0M
      CPU: 893ms
   CGroup: /system.slice/httpd.service
           └─26345 /usr/sbin/httpd -DFOREGROUND
             └─26375 /usr/sbin/httpd -DFOREGROUND
               └─26378 /usr/sbin/httpd -DFOREGROUND
                 └─26381 /usr/sbin/httpd -DFOREGROUND
                   └─26391 /usr/sbin/httpd -DFOREGROUND

Jul 28 11:02:48 ip-10-0-1-117.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Jul 28 11:02:48 ip-10-0-1-117.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Jul 28 11:02:48 ip-10-0-1-117.ec2.internal httpd[26345]: Server configured, listening on: port 80
[ec2-user@ip-10-0-1-117 ~]$

```

Step 6: Verify the Public Website

Open a browser: <http://<Public-IP>>



Welcome to Public EC2

Apache Web Server is Running

Step 7: Connect to the Private EC2

Since the private instance has no public IP, connect through the public instance (acting as a bastion host).

From the **Public EC2**:

Copy your private key to the public instance if needed:

```
$ scp -i linux-testing.pem linux-testing.pem ec2-user@32.199.136.221:/home/ec2-user/
```

On the public instance: `$ ssh -i linux-testing.pem ec2-user@32.199.136.221`

```
$ chmod 400 linux-testing.pem
```

```
$ ssh -i linux-testing.pem ec2-user@10.0.2.141
```



```

ec2-user@ip-10-0-2-141:~
[ec2-user@ip-10-0-2-141 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-07-28 11:51:42 UTC; 2min 2s ago
     Docs: man:httpd.service(8)
  Main PID: 27608 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 177 (limit: 1059)
   Memory: 14.0M
      CPU: 211ms
   CGroup: /system.slice/httpd.service
           └─27608 /usr/sbin/httpd -DFOREGROUND
             └─27638 /usr/sbin/httpd -DFOREGROUND
               └─27639 /usr/sbin/httpd -DFOREGROUND
                 └─27640 /usr/sbin/httpd -DFOREGROUND
                   └─27641 /usr/sbin/httpd -DFOREGROUND

Jul 28 11:51:42 ip-10-0-2-141.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Jul 28 11:51:42 ip-10-0-2-141.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Jul 28 11:51:42 ip-10-0-2-141.ec2.internal httpd[27608]: Server configured, listening on: port 80
[ec2-user@ip-10-0-2-141 ~]$ |

```

Step 9: Create a Sample Web Page (Private EC2)

Step 10: Verify Apache on the Private EC2

From the private instance:

curl localhost

```

[ec2-user@ip-10-0-2-141 ~]$ sudo tee /var/www/html/index.html > /dev/null <<EOF
<html>
<head>
<title>Private EC2</title>
</head>
<body>
<h1>Welcome to Private EC2</h1>
<h2>Apache Web Server is Running</h2>
</body>
</html>
EOF
[ec2-user@ip-10-0-2-141 ~]$ curl localhost
<html>
<head>
<title>Private EC2</title>
</head>
<body>
<h1>Welcome to Private EC2</h1>
<h2>Apache Web Server is Running</h2>
</body>
</html>
[ec2-user@ip-10-0-2-141 ~]$ |

```

```
[ec2-user@ip-10-0-2-141 ~]$ sudo tee /var/www/html/index.html > /dev/null <<EOF
<html>
<head>
<title>Private EC2</title>
</head>
<body>
<h1>Welcome to Private EC2</h1>
<h2>Apache Web Server is Running</h2>
</body>
</html>
EOF
[ec2-user@ip-10-0-2-141 ~]$ curl localhost
<html>
<head>
<title>Private EC2</title>
</head>
<body>
<h1>Welcome to Private EC2</h1>
<h2>Apache Web Server is Running</h2>
</body>
</html>
[ec2-user@ip-10-0-2-141 ~]$ curl 10.0.2.141
<html>
<head>
<title>Private EC2</title>
</head>
<body>
<h1>Welcome to Private EC2</h1>
<h2>Apache Web Server is Running</h2>
</body>
</html>
[ec2-user@ip-10-0-2-141 ~]$
```

```
ec2-user@ip-10-0-2-141:~
[ec2-user@ip-10-0-2-141 ~]$ ping www.amazon.com
PING cf.47cf2c8c9-frontier.amazon.com (3.170.42.148) 56(84) bytes of data:
64 bytes from server-3-170-42-148.iad61.r.cloudfront.net (3.170.42.148): icmp_seq=1 ttl=248 time=1.53 ms
64 bytes from server-3-170-42-148.iad61.r.cloudfront.net (3.170.42.148): icmp_seq=2 ttl=248 time=1.10 ms
64 bytes from server-3-170-42-148.iad61.r.cloudfront.net (3.170.42.148): icmp_seq=3 ttl=248 time=1.15 ms
64 bytes from server-3-170-42-148.iad61.r.cloudfront.net (3.170.42.148): icmp_seq=4 ttl=248 time=1.10 ms
64 bytes from server-3-170-42-148.iad61.r.cloudfront.net (3.170.42.148): icmp_seq=5 ttl=248 time=1.09 ms
64 bytes from server-3-170-42-148.iad61.r.cloudfront.net (3.170.42.148): icmp_seq=6 ttl=248 time=1.15 ms
64 bytes from server-3-170-42-148.iad61.r.cloudfront.net (3.170.42.148): icmp_seq=7 ttl=248 time=1.08 ms
64 bytes from server-3-170-42-148.iad61.r.cloudfront.net (3.170.42.148): icmp_seq=8 ttl=248 time=1.09 ms
64 bytes from server-3-170-42-148.iad61.r.cloudfront.net (3.170.42.148): icmp_seq=9 ttl=248 time=1.11 ms
```

8.Create an Application Load Balancer (ALB) and Attach Both EC2 Instances

Prerequisites

Ensure you have:

- ☒ Public EC2 (Apache installed)
- ☒ Private EC2 (Apache installed)
- ☒ VPC
- ☒ Public and Private Subnets
- ☒ Internet Gateway
- ☒ NAT Gateway

Step 1: Create a Target Group

1. Open EC2 Console.

2. Click **Target Groups** → **Create Target Group**.

Configure:

Field	Value
Target Type	Instances
Name	Apache-TG
Protocol	HTTP
Port	80
VPC	My-VPC

Click **Next**.

EC2 > Target groups > Create target group

Indicate what resource type you want to target. Only the selected resource type can be registered to this target group.

- ☒ **Instances**
Supports load balancing to instances in a VPC. Integrate with Auto Scaling Groups or ECS services for automatic management.
Suitable for: ALB NLB GWLB
- ☐ **IP addresses**
Supports load balancing to VPC and on-premises resources. Facilitates routing to IP addresses and network interfaces on the same instance. Supports IPv6 targets.
Suitable for: ALB NLB GWLB
- ☐ **Lambda function**
Supports load balancing to a single Lambda function. ALB required as traffic source.
Suitable for: ALB
- ☐ **Application Load Balancer**
Allows use of static IP addresses and PrivateLink with an Application Load Balancer. NLB required as traffic source.
Suitable for: NLB

Target group name
Name must be unique per Region per AWS account.
Apache-TargetGroup
Accepts: a-z, A-Z, 0-9, and hyphen (-). Can't begin or end with hyphen. 1-32 total characters; Count: 18/32

Protocol
Protocol for communication between the load balancer and targets.
HTTP

Port
Port number where targets receive traffic. Can be overridden for individual targets during registration.
80
1-65535

IP address type
Only targets with the indicated IP address type can be registered to this target group.
☒ **IPv4**
Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.

Step 2: Register Targets

1. Select:
 - Public-EC2
 - Private-EC2
2. Click **Include as pending below**.
3. Click **Create Target Group**.

2 selections are now pending below. Include more or register targets when ready.

Review targets

Targets (2)

Remove all pending

Show only pending

< 1 >

Instance ID	Name	Port	State	Security groups	Zone	Private IPv4 address	Subnet ID
i-0add56ee021dad5e3	pri-EC2	80	Running	pri-SG	us-east-1b	10.0.2.141	subnet-0f8aa038baf2fa2a2
i-0fc5b41e8274ec108	pub-EC2	80	Running	public-SG	us-east-1a	10.0.1.117	subnet-018a57c892d9d91bb

2 pending

Cancel

Previous

Next

EC2 > Target groups > Apache-TargetGroup

Successfully created the target group: Apache-TargetGroup. Anomaly detection is automatically applied to all registered targets. Results can be viewed in the Targets tab.

Give feedback

Actions

Apache-TargetGroup

Details

Target type

Instance

Protocol : Port

HTTP: 80

Protocol version

HTTP1

VPC

[vpc-0f5f3d60909fb0aa7](#)

IP address type

IPv4

Load balancer

[None associated](#)

2

0

0

2

0

0

Total targets

Healthy

Unhealthy

Unused

Initial

Draining

0 Anomalous

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Step 3: Create Application Load Balancer

- Go to **Load Balancers**.
- Click **Create Load Balancer**.
- Select **Application Load Balancer**.

Configure:

Field	Value
Name	Apache-ALB
Scheme	Internet-facing
IP Address Type	IPv4
VPC	My-VPC

Field	Value
Subnets	Public-Subnet-1 and Public-Subnet-2

Step 4: Configure Security Group

Create or select a security group with:

Type Port Source

HTTP 80 0.0.0.0/0

Step 5: Configure Listener

- Protocol: HTTP
- Port: 80

Forward to:

- Apache-TG

Click **Create Load Balancer**.

The screenshot shows the AWS Management Console interface for the 'Load balancers' section. The left sidebar contains navigation links for EC2, Instances, Images, Elastic Block Store, and Network & Security. The main content area displays a table of load balancers with one entry, 'Apache-ALB', which is in an 'Active' state. Below the table, the configuration details for 'Apache-ALB' are shown, including its type (Application), scheme (Internet-facing), VPC (vpc-0f5f3d60909fb0aa7), availability zones (us-east-1a and us-east-1b), and the date it was created (July 28, 2026).

Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones
Apache-ALB	Active	application	Internet-facing	IPv4	vpc-0f5f3d60909fb0aa7	2 Availability Zones

Load balancer: Apache-ALB

Load balancer type Application	Status Active	VPC vpc-0f5f3d60909fb0aa7	Load balancer IP address type IPv4
Scheme Internet-facing	Hosted zone Z35SXDOTRQ7X7K	Availability Zones subnet-018a57c892d9d91bb us-east-1a (use1-az1) subnet-02430ba38f20e7275 us-east-1b (use1-az2)	Date created July 28, 2026, 17:57 (UTC+05:30)

Load balancer ARN
 arn:aws:elasticloadbalancing:us-east-1:571833381790:loadbalancer/app/Apache-ALB/

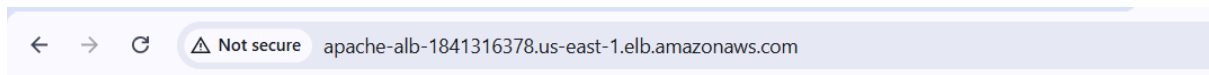
DNS name
 Apache-ALB-1841316378.us-east-1.elb.amazonaws.com (A Record)

Step 6: Verify

Wait until the ALB status is **Active**.

Open:

<http://<ALB-DNS-Name>>: Apache-ALB-1841316378.us-east-1.elb.amazonaws.com



Welcome to Public EC2

Apache Web Server is Running

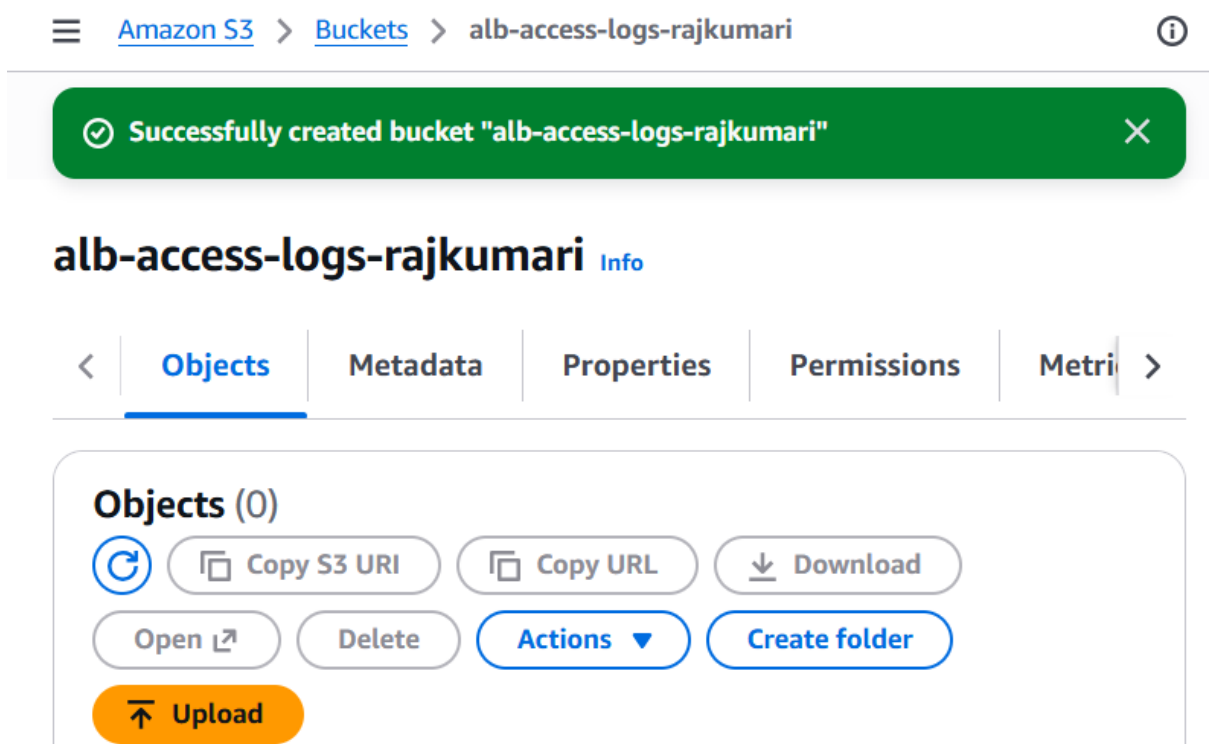
9.Store application load balancer ALB logs in S3.

Step 1: Create an S3 Bucket

Example:

- Bucket Name:
 - alb-access-logs-rajkumari
- Region:
 - N. Virginia (us-east-1)

Keep default settings and create the bucket.



Step 2: Enable ALB Access Logs

1. Open EC2 → Load Balancers.
2. Select Apache-ALB.

3. Go to the **Attributes** tab.

4. Click **Edit**.

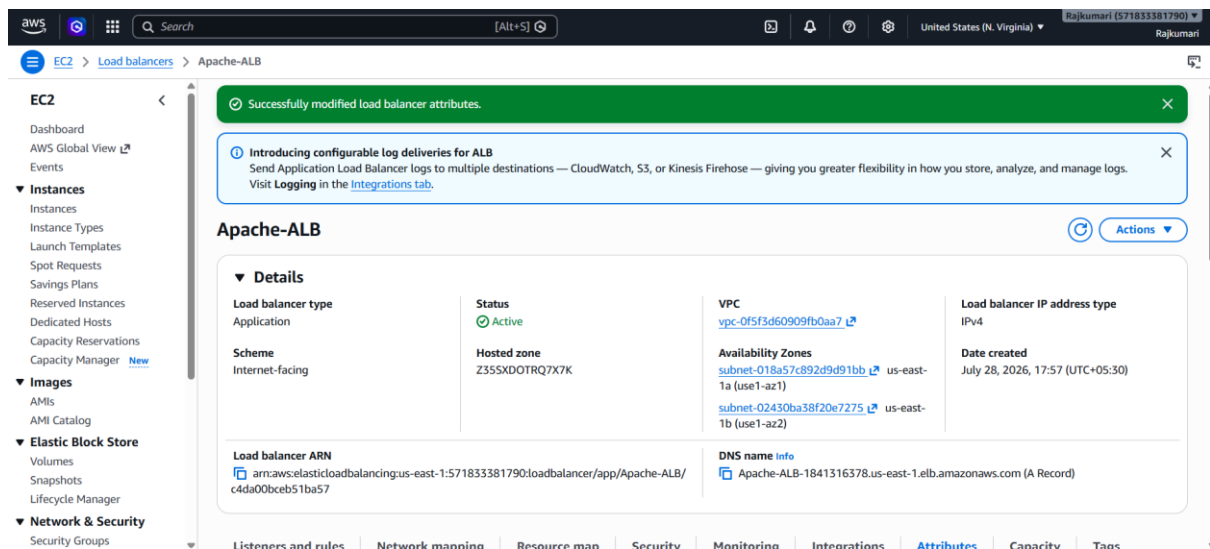
Enable:

- **Access Logs** → **On**

Choose:

- S3 Bucket:
 - alb-access-logs-rajkumari

Save changes.



Step 3: Generate Traffic

Open the ALB DNS several times:

http://<ALB-DNS>: Apache-ALB-1841316378.us-east-1.elb.amazonaws.com



Welcome to Private EC2

Apache Web Server is Running

Step 4: Verify

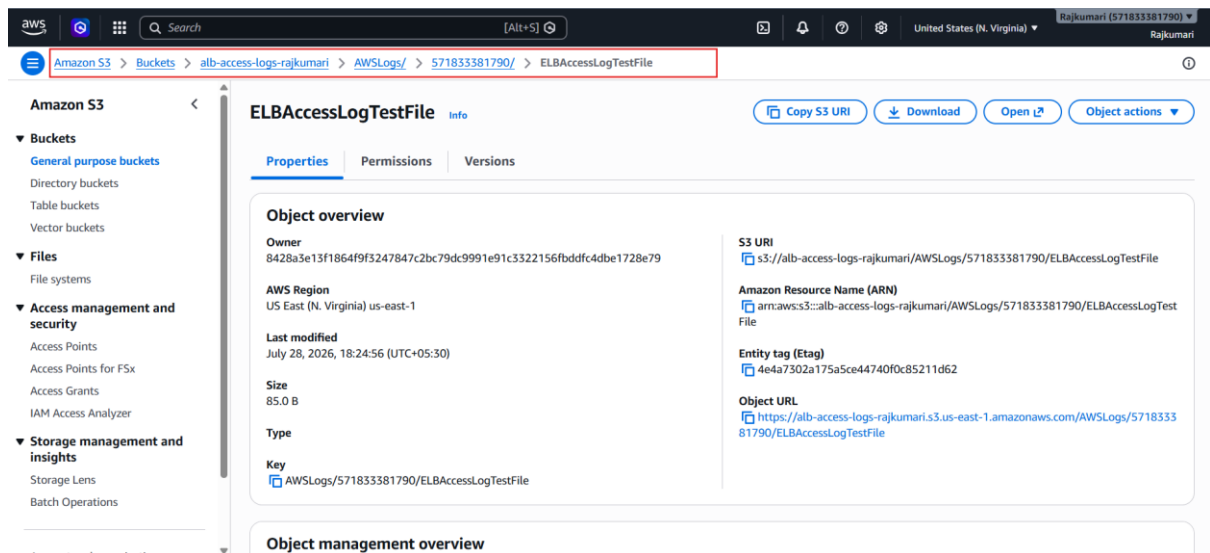
After a few minutes, go to your S3 bucket.

You should see log files under a path similar to:

AWSLogs/

Account-ID/

elasticloadbalancing/



10.Store the VPC flow logs in a CloudWatch log group.

Step 1: Create a CloudWatch Log Group

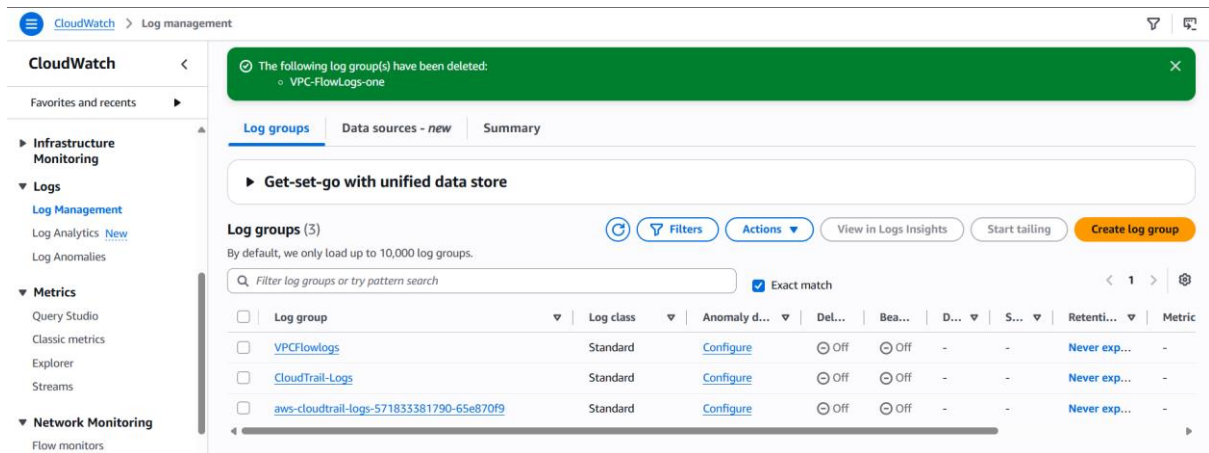
1. Open CloudWatch.
2. Click **Log Groups**.
3. Click **Create Log Group**.

Example:

Field Value

Name VPC-FlowLogs

Click **Create**.



Step 2: Create an IAM Role

1. Open IAM.
2. Create a role.

Configuration:

Field **Value**

Trusted Entity AWS Service

Service VPC Flow Logs

Attach policy:

- CloudWatchLogsFullAccess

Role Name:

- VPCFlowLogsRole

Create the role.

Roles (13) Info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Role name	Trusted entities	Last activity
EC2S3Role	AWS Service: ec2	6 days ago
s3crr_role_for_rajkumari-s3-2026	AWS Service: s3	5 days ago
VPCFlowLogsRole	AWS Service: vpc-flow-logs	5 days ago

Roles Anywhere Info

Authenticate your non AWS workloads and securely provide access to AWS services.

Access AWS from your non AWS workloads

Operate your non AWS workloads using the same authentication and authorization strategy that you use within AWS.

X.509 Standard

Use your own existing PKI infrastructure or use [AWS Certificate Manager Private Certificate Authority](#) to authenticate identities.

Temporary credentials

Use temporary credentials with ease and benefit from the enhanced security they provide.

VPCFlowLogsRole Info

Summary

Creation date: July 16, 2026, 10:05 (UTC+05:30)

Last activity: 5 days ago

ARN: [arn:aws:iam::571833381790:role/VPCFlowLogsRole](#)

Maximum session duration: 1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (1) Info

You can attach up to 10 managed policies.

Policy name	Type	Attached entities
CloudWatchLogsFullAccess	AWS managed	1

Permissions boundary (not set)

Step 3: Enable Flow Logs

1. Open **VPC Console**.
2. Select **Your VPCs**.
3. Select **My-VPC**.
4. Open the **Flow Logs** tab.
5. Click **Create Flow Log**.

Configure:

Field **Value**

Filter All

Destination Send to CloudWatch Logs

Field Value

Log Group VPC-FlowLogs

IAM Role VPCFlowLogsRole

Click **Create Flow Log**.

The screenshot shows the AWS VPC console with a green notification banner at the top stating: "Successfully created flow log for the following resource: vpc-0f5f3d60909fb0aa7". Below this, the details for flow log **fl-03c5be8418bf1cce1** are displayed. The details are organized into a grid:

Details Flow Log ID fl-03c5be8418bf1cce1	Destination Type cloud-watch-logs	Traffic Type All	File Format -
Name -	Destination Name VPCFlowlogs	Max Aggregation Interval 10 minutes	Hive Compatible Partitions -
State Active	IAM Role arn:aws:iam::571833381790:role/VPCFlowLogsRole	Log Format Default	Partition Logs -
Creation Time Tuesday, July 28, 2026 at 19:34:03 GMT+5:30	Cross Account IAM Role -	Tag field specifications -	

Below the details, there are tabs for "Tags" and "Integrations". The "Tags" tab is active, showing a search bar and a table with columns "Key" and "Value".

Step 4: Verify

Generate some network traffic (for example, access the ALB or SSH into an instance).

Go to:

CloudWatch → Log Groups → VPC-FlowLogs

You should see log streams containing VPC network traffic.

The screenshot shows the AWS CloudWatch console. The left sidebar contains navigation links for "Infrastructure Monitoring", "Logs", "Metrics", and "Network Monitoring". The "Logs" section is expanded, showing "Log Management", "Log Analytics", and "Log Anomalies". The main content area displays "Log events" for the log group **eni-019ece6e082eff790-all**. At the top, there are buttons for "Actions", "Start tailing", and "Create metric filter". Below these is a search bar and a "Display" button. The log events are listed in a table with columns "Timestamp" and "Message".

Timestamp	Message
2026-07-28T14:05:52.000Z	2 571833381790 eni-019ece6e082eff790 10.0.1.117 10.0.3.75 80 44392 6 5 676 1785247552 1785247583 ACCEPT OK
2026-07-28T14:05:52.000Z	2 571833381790 eni-019ece6e082eff790 10.0.1.117 10.0.3.75 80 44392 6 5 676 1785247552 1785247583 ACCEPT OK
2026-07-28T14:05:52.000Z	2 571833381790 eni-019ece6e082eff790 66.132.186.141 10.0.1.117 16820 10364 6 1 60 1785247552 1785247583 REJECT OK
2026-07-28T14:05:52.000Z	2 571833381790 eni-019ece6e082eff790 10.0.1.117 103.160.27.187 22 50911 6 1 40 1785247552 1785247583 ACCEPT OK
2026-07-28T14:05:52.000Z	2 571833381790 eni-019ece6e082eff790 85.217.140.53 10.0.1.117 58723 35277 6 1 52 1785247552 1785247583 REJECT OK
2026-07-28T14:05:52.000Z	2 571833381790 eni-019ece6e082eff790 10.0.3.75 10.0.1.117 44392 80 6 5 393 1785247552 1785247583 ACCEPT OK
2026-07-28T14:05:52.000Z	2 571833381790 eni-019ece6e082eff790 101.68.215.26 10.0.1.117 0 0 47 1 66 1785247552 1785247583 REJECT OK
2026-07-28T14:05:52.000Z	2 571833381790 eni-019ece6e082eff790 10.0.1.197 10.0.1.117 64174 80 6 5 393 1785247552 1785247583 ACCEPT OK
2026-07-28T14:05:52.000Z	2 571833381790 eni-019ece6e082eff790 152.115.152.218 10.0.1.117 0 0 47 1 66 1785247552 1785247583 REJECT OK

11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

Objective

Create a **CloudWatch Dashboard** to monitor:

- EC2 CPU Utilization
 - Apache (httpd) service status
-

Prerequisites

Make sure you have:

- ☒ EC2 Public Instance
- ☒ EC2 Private Instance
- ☒ Apache (httpd) installed on both instances
- ☒ CloudWatch Agent installed on both instances

Note: CPU Utilization is available by default in CloudWatch. However, **Apache service status is not available by default**. You need the **CloudWatch Agent** configured to collect procstat metrics for the httpd process.

Part 1: Monitor CPU Utilization

Step 1: Open CloudWatch

1. Login to AWS Console.
2. Search for **CloudWatch**.
3. Open **CloudWatch**

Step 2: Create Dashboard

1. Click **Dashboards**.
2. Click **Create dashboard**.

Enter:

Field	Value
Dashboard Name	EC2-Monitoring

Click **Create dashboard**.

Step 3: Add CPU Widget

1. Select **Line**.
2. Click **Next**.

Navigate:

AWS/EC2



Per-Instance Metrics



CPUUtilization

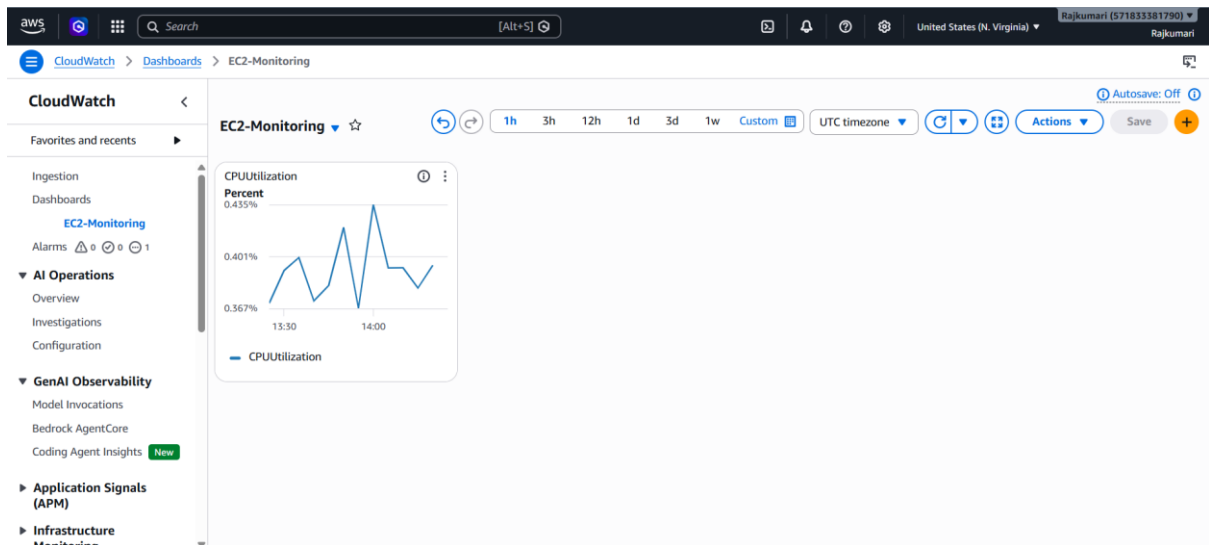
Select:

- Public EC2
- Click **Create widget**.

Step 4: Save Dashboard

Click **Save Dashboard**.

You will now see CPU utilization for both EC2 instances.



Part 2: Monitor Apache Service

Step 1: Verify CloudWatch Agent

On both EC2 instances:

Install CloudWatch Agent on Public EC2

Run: `$ sudo yum install amazon-cloudwatch-agent -y`

`$ sudo systemctl start amazon-cloudwatch-agent`

`$ sudo systemctl status amazon-cloudwatch-agent`

```
[ec2-user@ip-10-0-1-117 ~]$ sudo systemctl enable amazon-cloudwatch-agent
[ec2-user@ip-10-0-1-117 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-07-28 11:02:48 UTC; 3h 37min ago
     Docs: man:httpd.service(8)
  Main PID: 26345 (httpd)
    Status: "Total requests: 538; Idle/Busy workers 100/0; Requests/sec: 0.0412"
     Tasks: 230 (limit: 1059)
    Memory: 20.4M
       CPU: 14.224s
    CGroup: /system.slice/httpd.service
            └─26345 /usr/sbin/httpd -DFOREGROUND
              └─26375 /usr/sbin/httpd -DFOREGROUND
                └─26378 /usr/sbin/httpd -DFOREGROUND
                  └─26381 /usr/sbin/httpd -DFOREGROUND
                    └─26391 /usr/sbin/httpd -DFOREGROUND
                      └─27647 /usr/sbin/httpd -DFOREGROUND

Jul 28 11:02:48 ip-10-0-1-117.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server.
Jul 28 11:02:48 ip-10-0-1-117.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Jul 28 11:02:48 ip-10-0-1-117.ec2.internal httpd[26345]: Server configured, listening on: port 80
lines 1-20/20 (END)...skipping...
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-07-28 11:02:48 UTC; 3h 37min ago
     Docs: man:httpd.service(8)
  Main PID: 26345 (httpd)
    Status: "Total requests: 538; Idle/Busy workers 100/0; Requests/sec: 0.0412; Bytes served/sec: 22 B/sec"
     Tasks: 230 (limit: 1059)
    Memory: 20.4M
       CPU: 14.224s
    CGroup: /system.slice/httpd.service
            └─26345 /usr/sbin/httpd -DFOREGROUND
              └─26375 /usr/sbin/httpd -DFOREGROUND
                └─26378 /usr/sbin/httpd -DFOREGROUND
                  └─26381 /usr/sbin/httpd -DFOREGROUND
                    └─26391 /usr/sbin/httpd -DFOREGROUND
                      └─27647 /usr/sbin/httpd -DFOREGROUND

Jul 28 11:02:48 ip-10-0-1-117.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server.
Jul 28 11:02:48 ip-10-0-1-117.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Jul 28 11:02:48 ip-10-0-1-117.ec2.internal httpd[26345]: Server configured, listening on: port 80
```

Step 2: Check the Agent Configuration

Your CloudWatch Agent configuration should include procstat for Apache.

Example configuration:

```
{  
  "metrics": {  
    "metrics_collected": {  
      "procstat": [  
        {  
          "pattern": "httpd",  
          "measurement": [  
            "pid_count"  
          ]  
        }  
      ]  
    }  
  }  
}
```

Step 3: Add Apache Widget

Go to:

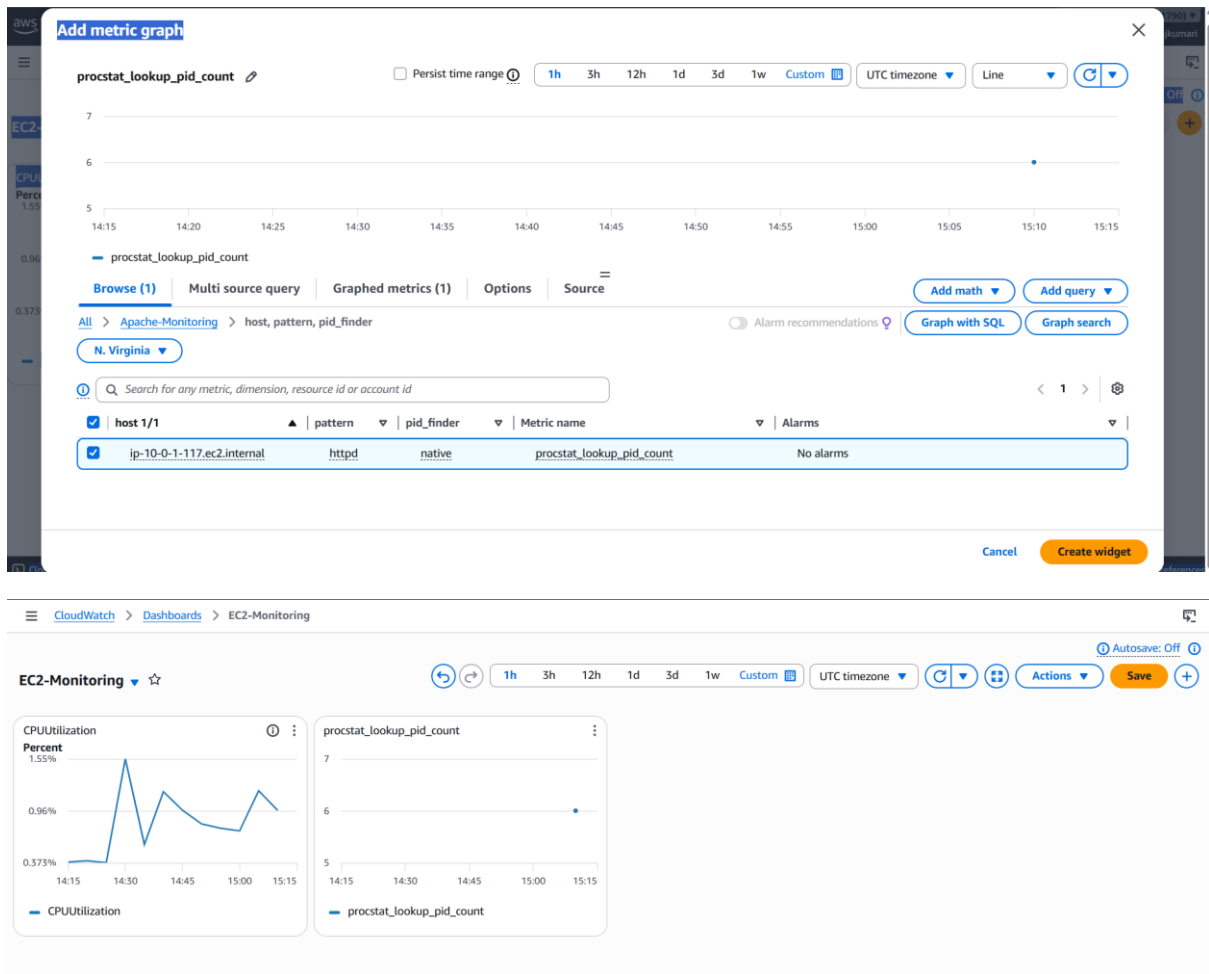
CloudWatch → Dashboards → EC2-Monitoring

Click:

Add Widget

Choose:

Line



12. If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

Objective: When the EC2 instance CPU utilization goes above 70%, a CloudWatch alarm should trigger an **Auto Scaling Group (ASG)** to launch a new EC2 instance automatically.

Step 1: Create Launch Template

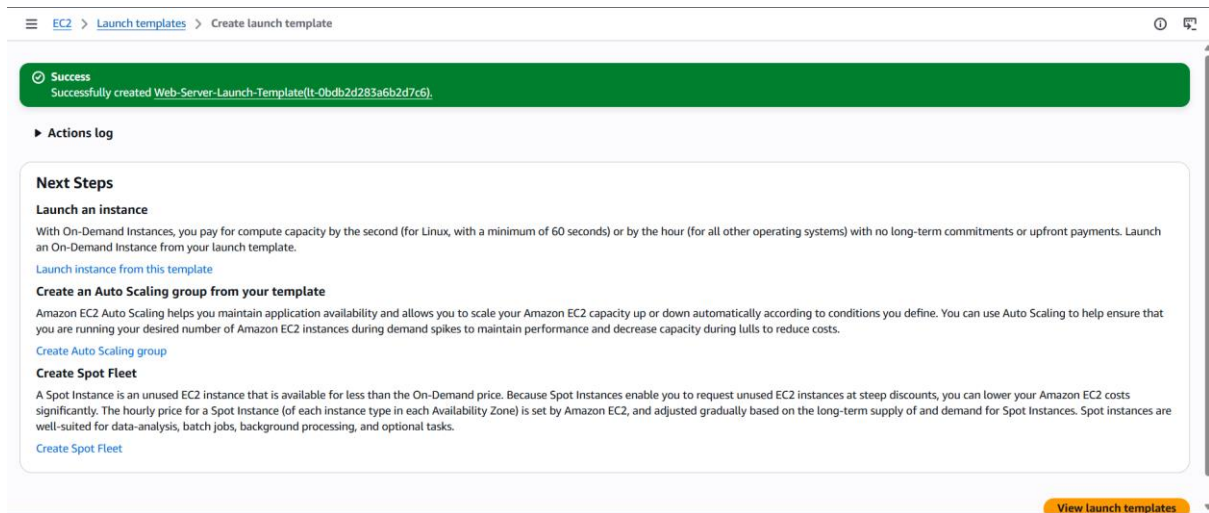
A Launch Template contains the EC2 instance configuration that Auto Scaling uses to launch new instances.

Navigate:

AWS Console → EC2 → Launch Templates → Create Launch Template

Configure:

Launch Template Name



Step 2: Create Target Group

The Target Group is used by the Load Balancer to route traffic to EC2 instances.

Navigate:

EC2 Console



Target Groups



Create Target Group

Select:

Instances

Configuration:

Target Group Name

Web-TG

Protocol:

HTTP

Port:

80

Health Check:

HTTP

Path:

/

Click:

Create Target Group

The screenshot displays the AWS Management Console interface for a target group named 'Web-TG'. At the top, a green notification bar states: 'Successfully created the target group: Web-TG. Anomaly detection is automatically applied to all registered targets. Results can be viewed in the Targets tab.' Below this, the 'Web-TG' header is visible with an 'Actions' dropdown menu. The 'Details' section shows the ARN 'arn:aws:elasticloadbalancing:us-east-1:571833381790:targetgroup/Web-TG/2a4ffb50fdc90152'. It lists 'Target type' as 'Instance', 'IP address type' as 'IPv4', 'Protocol : Port' as 'HTTP: 80', 'Protocol version' as 'HTTP1', and 'VPC' as 'vpc-0f5f3d60909fb0aa7'. A summary row indicates '0 Total targets', '0 Healthy', '0 Unhealthy', '0 Unused', '0 Initial', and '0 Draining'. Below the details, tabs for 'Targets', 'Monitoring', 'Health checks', 'Attributes', and 'Tags' are present. The 'Targets' tab is active, showing 'Registered targets (0)' with an 'Info' link. A note states: 'Anomaly mitigation: Not applicable'. At the bottom, there are 'Deregister' and 'Register targets' buttons. A descriptive text at the very bottom explains that target groups route requests to individual registered targets using the protocol and port number specified, and that health checks are performed on all registered targets according to the target group's health check settings.

Step 3: Create Application Load Balancer

Navigate:

EC2

↓

Load Balancers

↓

Create Load Balancer

Select:

Application Load Balancer

Configuration:

Name:

Web-ALB

Scheme:

Internet-facing

IP Type:

IPv4

Network Mapping

Select:

Your VPC

Choose:

Two Public Subnets

Security Group

Allow:

HTTP - Port 80

Listener

Protocol:

HTTP

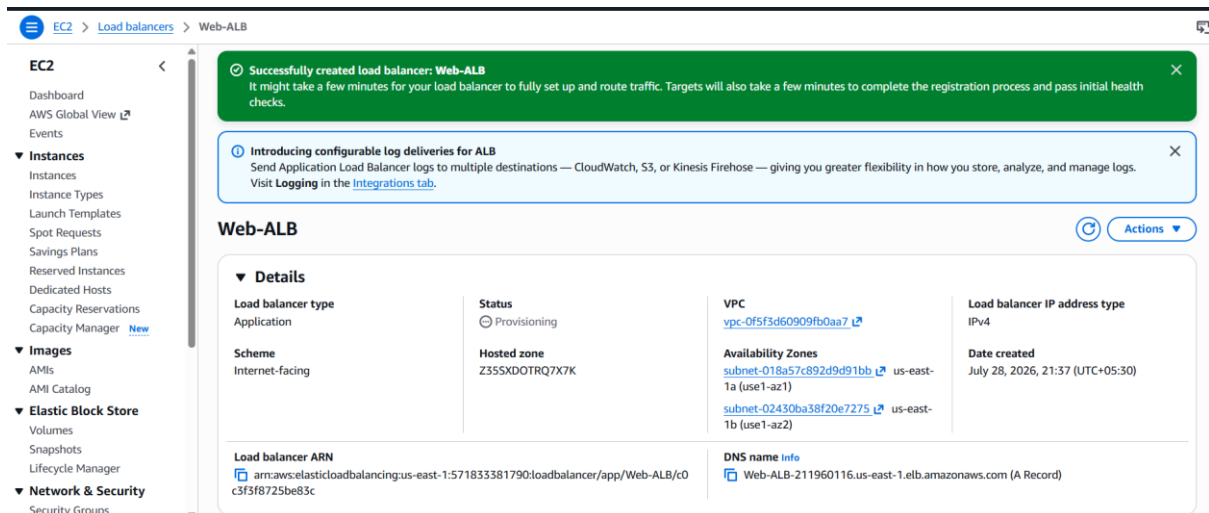
Port:

80

Forward to:

Web-TG

Create Load Balancer.



Step 4: Create Auto Scaling Group

Navigate:

EC2



Auto Scaling Groups



Create Auto Scaling Group

Step 4.1 Select Launch Template

Choose:

Web-Server-Launch-Template

Click:

Next

Step 4.2 Configure Network

Select:

Your VPC

Choose:

Private Subnets

(Recommended for production)

Step 4.3 Attach Load Balancer

Select:

Attach to an existing Load Balancer

Choose:

Web-TG

Enable:

Elastic Load Balancing health checks

Step 4.3 Attach Load Balancer

Select:

Attach to an existing Load Balancer

Choose:

Web-TG

Enable:

Elastic Load Balancing health checks

Step 4.4 Configure Group Size

Example:

Desired capacity:

1

Minimum capacity:

1

Maximum capacity:

3

Meaning:

- Minimum 1 instance always running
- Can scale up to 3 instances

Step 6: Verify CloudWatch Alarm

AWS automatically creates a CloudWatch alarm.

Navigate:

CloudWatch



Alarms



All Alarms

You will see alarms similar to:

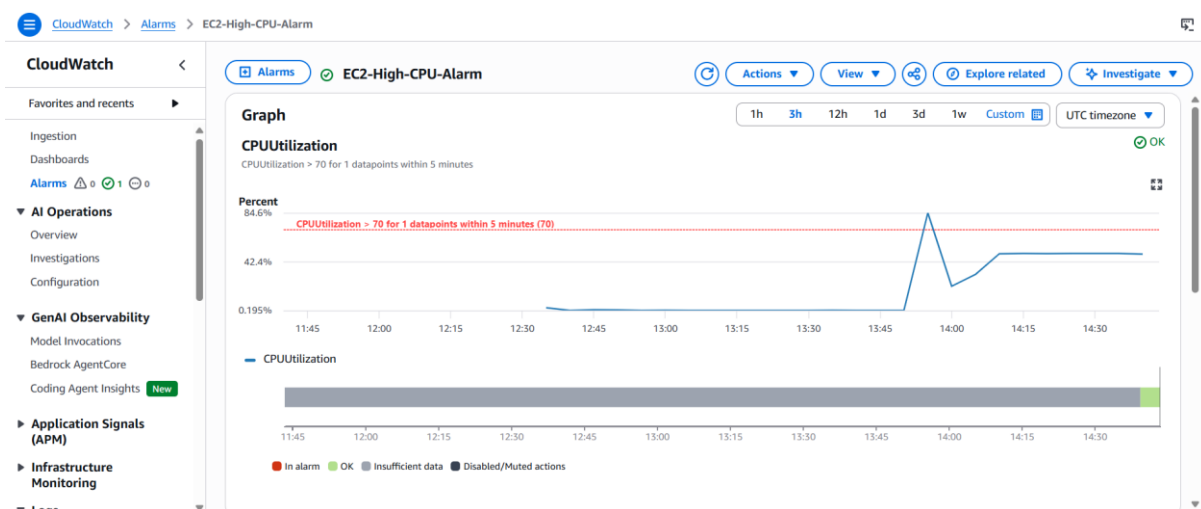
TargetTracking-CPU-70

Condition:

CPUUtilization $\geq$ 70%

Action:

Increase Auto Scaling Group capacity



EC2

Instances

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Successfully attached CloudWatchAgentRole to instance i-Ofc5b41e8274ec108

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Instances (5)

Info

Find Instance by attribute or tag (case-sensitive)

Instance state = running

Clear filters

1

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Put
☐		i-020f281a19ce9bba5	Running	t3.micro	3/3 checks passed	us-east-1c	ec2-54-235-28-116.co...	54.
☐	pri-EC2	i-0add56ee021dad5e3	Running	t3.micro	3/3 checks passed	us-east-1b	-	-
☐		i-0e051e69b08a37e41	Running	t3.micro	3/3 checks passed	us-east-1b	-	-
☐		i-03c98d7671c4a9da9	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-3-231-156-249.co...	3.2
☐	pub-EC2	i-Ofc5b41e8274ec108	Running	t3.micro	3/3 checks passed	us-east-1a	-	32.

All instances

Instance alarm states

In alarm

None

Show instances in alarm

Service health

EC2 service status

Operating normally

Health events affecting EC2 instances

Open events

None

Show instances

Upcoming events

None

Show instances